

Asserten — Technical Reference

May 30, 2026

CONTENTS

asserten	
⚠ It needs a backend	
Install	
The flow (10-minute demo)	
Command reference (21 commands)	
How it's built	
The backend API (what the client calls)	
Limits in v0.1	
Development	

asserten

A **Claude Code plugin** that takes any LLM agent and shows you how to make it measurably more reliable — by auditing its prompt, generating a behavioral test suite, optimizing it, and gating deploys on the scenarios you care about. Every step is an `/asserten-*` slash command.

It produces four versions of your agent, side by side, with the pass-rate delta on a fresh test suite for each:

version	what it is
v0	the prompt you brought, untouched
v1	v0 + the audit patches you accepted
v2a	LIGHT optimization — pick-best across K candidate v1s (free, instant)
v2b	DEEP optimization — full cross-refine (~15 min, ~\$10–20 of LLM cost)

⚠ It needs a backend

asserten is a **thin client**. All the heavy lifting (audit, test generation, eval, judges, optimization) runs in a separate **agentops-backend** service that holds the LLM keys. The plugin just drives that backend over HTTP.

So to actually run anything you need two things:

```
export ASSERTEN_BACKEND_URL=https://<your-backend-host> # default http://local-
host:8000
export ASSERTEN_API_KEY=<the-key-you-were-given> # per-prospect; see
below
```

You never put your Anthropic/OpenAI keys in the plugin — the backend operator holds those. Without a reachable backend, the commands install fine but every call will fail to connect.

About the key. `ASSERTEN_API_KEY` is a per-prospect key the operator issues you. It's capped (default **3 agents** — i.e. you can take three different agents through the flow). Reads are open; once you've created your 3 agents, `/asserten-ingest` -and-commit returns `402 quota exhausted` and you ask for a fresh key. The key only gates *your* usage of the operator's backend — it's not an LLM key and never leaves the `X-Asserten-Key` header.

For the operator (issuing keys). From the agentops-backend host:

```
python -m scripts.api_keys_admin mint --label alice@acme --agents 3 # prints
the key once
python -m scripts.api_keys_admin list # keys +
caps
python -m scripts.api_keys_admin usage # tokens +
$ cost per key
python -m scripts.api_keys_admin revoke --label alice@acme
```

Counting and the cap are enforced automatically; minting/revoking is the only manual step. Every LLM call is logged with model + input/output tokens + USD cost attributed to the calling key, so `usage` shows exactly what each prospect spent. Your own `ASSERTEN_API_KEY` (the backend's env value) is the uncapped admin/master key — keep it for yourself.

Install

Via the plugin marketplace (recommended):

```
/plugin marketplace add vinsing09/asserten
/plugin install asserten@asserten
```

Developer install (clone + symlink):

```
git clone https://github.com/vinsing09/asserten ~/Documents/my_projects/asserten
cd ~/Documents/my_projects/asserten && pip install -e .
mkdir -p ~/.claude/plugins && ln -s "$PWD" ~/.claude/plugins/asserten
```

Restart Claude Code; type `/asserten` to confirm it loaded.

The flow (10-minute demo)

```

/asserten-ingest examples/sample_agent.json # 1. hand over the agent → draft
/asserten-audit                             # 2. see suggested prompt patches
/asserten-select all                         # 3. accept patches → creates v0 +
v1
/asserten-prepare-eval                      # 4. generate contract + test
cases on v1                                #   (optional: {"count": 75};
                                           #   default 40)
/asserten-eval v0                           # 5. eval the raw agent
/asserten-eval v1                           # 6. eval the audited agent
/asserten-compare                           # 7. v0 vs v1 pass-rate table

```

Then optionally optimize and gate:

```

/asserten-optimize-deep <eval_run_id>      # DEEP optimize v1 → v2b (~15
min)
/asserten-eval v2b
/asserten-scenarios v1                     # outcome dashboard (✓/x per sce-
nario)
/asserten-approve all-passing              # promote green cases to the
guaranteed set
/asserten-deploy-gate {"candidate": "v2b"} # block the deploy if v2b re-
gresses on them

```

Or run the whole core flow with one command: `/asserten-run`
`examples/sample_agent.json`.

Step 4 matters. `/asserten-prepare-eval` generates the contract and the test set; eval can't run without it. The set size defaults to 40 (15 is too small and inflates pass rates; very high counts hit a redundancy ceiling). Override per run with `/asserten-prepare-eval {"count": 60}`.

Command reference (21 commands)

Setup

command	what it does
<code>/asserten</code>	help / overview
<code>/asserten-ingest <path></code>	upload an agent (system prompt + tool schemas + goal) → draft
<code>/asserten-status</code>	show the current session (what's ingested / eval'd / optimized)
<code>/asserten-reset</code>	clear the session to start on a different agent

Build the contract & test set

command	what it does
<code>/asserten-audit</code>	run the audit → numbered list of suggested prompt patches
<code>/asserten-select all 1,3,5 none</code>	accept patches → creates v0 (raw) + v1 (audited)
<code>/asserten-prepare-eval</code>	generate the contract + auto test cases on v1 (<code>{"count": N}</code>)
<code>/asserten-show-contract</code>	show the extracted contract + mandate-coverage transparency

Bring your own test cases (optional)

command	what it does
<code>/asserten-byoe</code>	plain-English: <code>input</code> + what the agent should say / call / not do
<code>/asserten-add-tests <path></code>	full-schema JSON upload (for technical users)
<code>/asserten-skip-tests <ids></code> / <code>/asserten-unskip-tests <ids></code>	exclude / re-include cases

Evaluate & optimize

command	what it does
<code>/asserten-eval <v0 v1 v2a v2b></code>	run the full eval on a version
<code>/asserten-failures</code>	failing cases for the latest eval
<code>/asserten-optimize-light</code>	LIGHT optimize (pick-best across K v1s, 0 LLM cost) → v2a
<code>/asserten-optimize-deep <eval_run_id></code>	DEEP optimize (cross-refine, slow) → v2b
<code>/asserten-compare</code>	4-way pass-rate table with deltas

Outcome dashboard & deploy gate

command	what it does
<code>/asserten-scenarios <version></code>	tiles: ✓/✗ status per scenario from the latest eval
<code>/asserten-approve <ids all-passing> / /asserten-unapprove <ids></code>	manage the guaranteed set
<code>/asserten-deploy-gate {"candidate":"v2b"}</code>	BLOCK / PASS / INCONCLUSIVE vs a baseline on approved cases

Orchestration

command	what it does
<code>/asserten-run <path></code>	drive the whole flow, pausing at patch selection

How it's built

asserten/	the plugin (this repo, public)
├ .claude-plugin/	plugin + marketplace manifests
├ commands/ *.md	21 slash commands – each invokes the launcher
├ bin/asserten-cli	launcher: cd to plugin root + ensure deps + run
CLI	
├ client/	
│ └ cli.py	dispatch: `python -m client.cli <subcommand>`
│ └ api.py	AssertenClient – the HTTP calls to the backend
│ └ models.py	dataclasses (SessionState, GateResult, ...)
│ └ format.py	render markdown tables for chat output
│ └ session.py	per-run state in ~/.asserten/session.json
├ examples/sample_agent.json	a runnable demo agent
└ tests/	~185 unit tests

A slash command shells out to `"${CLAUDE_PLUGIN_ROOT}/bin/asserten-cli" <subcommand>`. The launcher `cd`s into the plugin root (so `client` imports regardless of the caller's working directory) and, on first run, creates a local `.venv` with `httpx` + `pydantic` if they aren't already importable — so a marketplace install works with no manual `pip install`. It then runs `python -m client.cli`, which calls `AssertenClient`, makes one HTTP request to the backend, and renders the result as a markdown table. **Session state** (`agent_id`, the four version ids, last eval pass-rates) lives in `~/.asserten/session.json` so each command knows what the previous step produced.

The backend API (what the client calls)

`X-Asserten-Key: <ASSERTEN_API_KEY>` is sent on mutating (POST) requests; GETs are open.

client method	endpoint
ingest	POST /agents/draft
audit	POST /agents/draft/{id}/audit
select	POST /agents/draft/{id}/commit
prepare-eval	POST /agents/{id}/versions/{v}/contract/generate + .../test-cases/generate?count=N
byoe / add / skip	POST .../test-cases/{byoe,user-provided,skip,unskip}
eval	POST /agents/{id}/versions/{v}/eval-runs
optimize	POST /agents/{id}/optimize/light · .../improvements/apply (DEEP, via job poll)
scenarios	GET /agents/{id}/versions/{v}/scenarios
approve	POST .../test-cases/{approve,unapprove}
deploy-gate	POST /agents/{id}/versions/{cand}/deploy-gate?baseline_version_id=X&strict=true

Example — ingest body (examples/sample_agent.json):

```
{
  "name": "Customer Support Agent (sample)",
  "raw_system_prompt": "You are a polite customer-support assistant...",
  "tool_schemas": [{"name": "lookup_order", "description": "...", "parameters": {...}}],
  "business_goal": "Resolve order/refund questions without over-refunding."
}
```

Example — deploy-gate response:

```
{
  "verdict": "BLOCKED",
  "approved_count": 11,
  "regressions": [{"scenario_name": "Refund within policy",
    "baseline_status": "passed", "candidate_status": "failed",
    "failure_origin": "agent"}],
  "judge_inconclusive": [], "stable_pass": [...], "reason": "1 approved scenario
regressed: ..."
}
```

Limits in v0.1

- **LIGHT optimize needs K pre-existing candidate v1s** — v0.1 doesn't auto-bulk-audit; re-use an audit study or set `candidate_v1_ids` in `~/.asserten/session.json`.
- **DEEP optimize wants the `eval_run_id`** from the prior `/asserten-eval v1` output.
- **Single active session** — one agent at a time; `/asserten-reset` to switch.
- **Deploy-gate** requires both versions to have an eval (or pass `auto_eval=true`, the default, to generate one on the fly).

Development

```
pip install -e ".[test]"
pytest -q # ~185 unit tests, ~2s
# E2E against a real backend:
ASSERTEN_E2E_BACKEND_URL=http://localhost:8000 \
  ASSERTEN_E2E_AGENT_ID=<id> ASSERTEN_E2E_VERSION_ID=<vid> \
  pytest tests/test_e2e_smoke.py -v
```

`PROJECT_KNOWLEDGE.md` has architecture decisions, layout, and chronological history.